SE LAB 5

NAME:SHIVAKUMARA GP

SRN:PES2UG24CS470

SEC:H

BEFORE FIXING ISSUES

1.bandit_report



```
L5_33_movie_reviews > bandit_report_before.txt
Run started:2026-10-14 06:33:35.989866+00:00

Test results:
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
   Severity: Medium   Confidence: Medium
   CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104_hardcoded_bind_all_interfaces.html
   Location: .\movie_reviews.py:4:14
3
4  STREAM_HOST = "0.0.0.0"
5  STREAM_PORT = 8000

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'reviews-demo'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\movie_reviews.py:6:21
5  STREAM_PORT = 8000
6  MODERATOR_PASSWORD = "reviews-demo"
7  movies = {}

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\movie_reviews.py:41:15
40     global featured
41     featured = random.choice(sorted(movies))
42     return featured

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: .\movie_reviews.py:46:11
45 def reviewId(title, reviewText):
46     return hashlib.md5((title + reviewText).encode()).hexdigest()[:10]
47

--------------------------------------------------

Code scanned:
   Total lines of code: 55
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

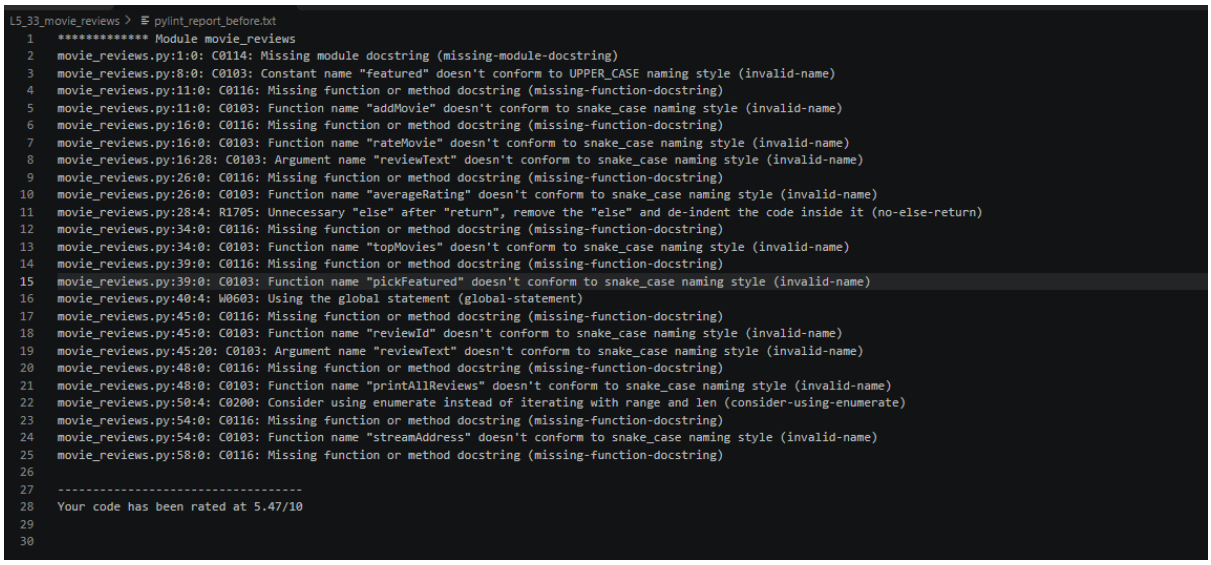
Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 2
      Medium: 1
      High: 1
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 2
      High: 2
Files skipped (0):
```

2.flake8_report



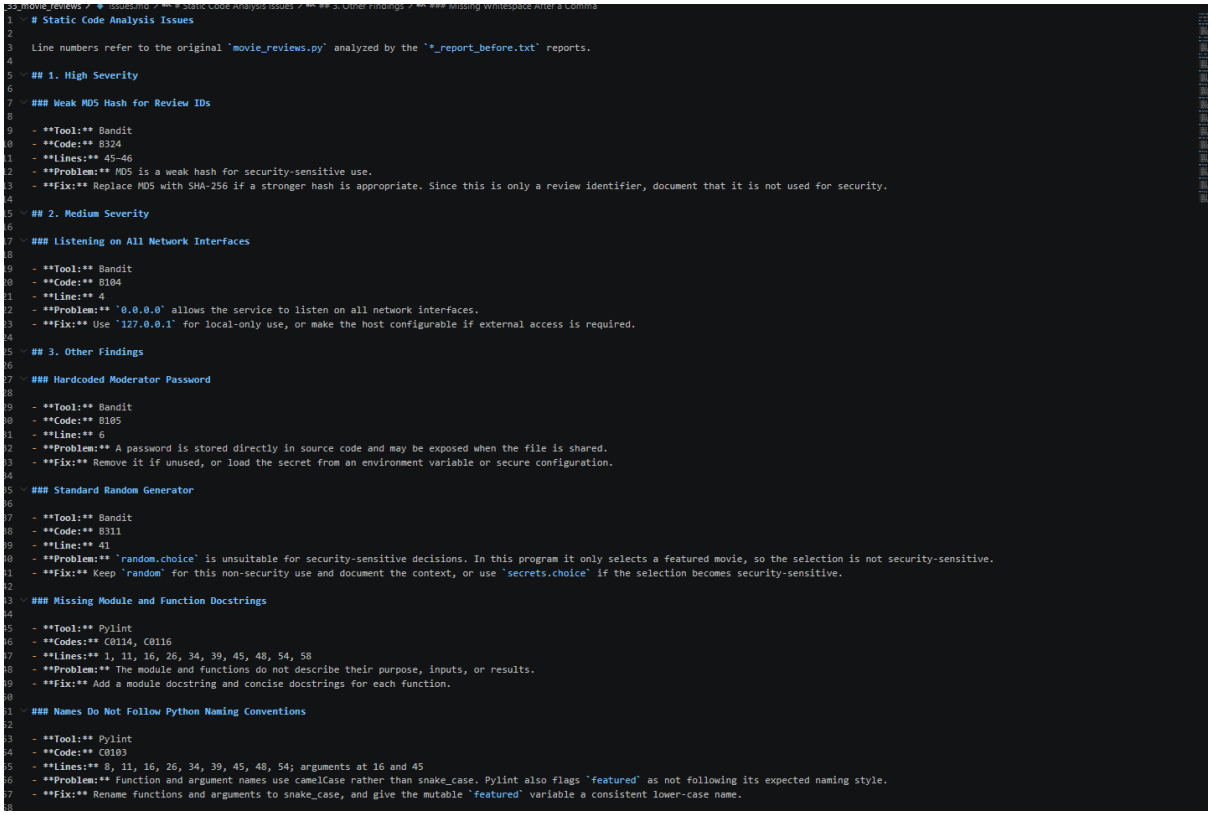
```
5_33_movie_reviews > flake8_report_before.txt
movie_reviews.py:35:11: E225 missing whitespace around operator
movie_reviews.py:48:1: E302 expected 2 blank lines, found 1
movie_reviews.py:70:67: E261 at least two spaces before inline comment
movie_reviews.py:72:20: E231 missing whitespace after ','
```

3.pylint_report



```
L5_33_movie_reviews > pylint_report_before.txt
1   ************* Module movie_reviews
2   movie_reviews.py:1:0: C0114: Missing module docstring (missing-module-docstring)
3   movie_reviews.py:8:0: C0103: Constant name "featured" doesn't conform to UPPER_CASE naming style (invalid-name)
4   movie_reviews.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)
5   movie_reviews.py:11:0: C0103: Function name "addMovie" doesn't conform to snake_case naming style (invalid-name)
6   movie_reviews.py:16:0: C0116: Missing function or method docstring (missing-function-docstring)
7   movie_reviews.py:16:0: C0103: Function name "rateMovie" doesn't conform to snake_case naming style (invalid-name)
8   movie_reviews.py:16:28: C0103: Argument name "reviewText" doesn't conform to snake_case naming style (invalid-name)
9   movie_reviews.py:26:0: C0116: Missing function or method docstring (missing-function-docstring)
10  movie_reviews.py:26:0: C0103: Function name "averageRating" doesn't conform to snake_case naming style (invalid-name)
11  movie_reviews.py:28:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
12  movie_reviews.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
13  movie_reviews.py:34:0: C0103: Function name "topMovies" doesn't conform to snake_case naming style (invalid-name)
14  movie_reviews.py:39:0: C0116: Missing function or method docstring (missing-function-docstring)
15  movie_reviews.py:39:0: C0103: Function name "pickFeatured" doesn't conform to snake_case naming style (invalid-name)
16  movie_reviews.py:40:4: W0603: Using the global statement (global-statement)
17  movie_reviews.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
18  movie_reviews.py:45:0: C0103: Function name "reviewId" doesn't conform to snake_case naming style (invalid-name)
19  movie_reviews.py:45:20: C0103: Argument name "reviewText" doesn't conform to snake_case naming style (invalid-name)
20  movie_reviews.py:48:0: C0116: Missing function or method docstring (missing-function-docstring)
21  movie_reviews.py:48:0: C0103: Function name "printAllReviews" doesn't conform to snake_case naming style (invalid-name)
22  movie_reviews.py:50:4: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
23  movie_reviews.py:54:0: C0116: Missing function or method docstring (missing-function-docstring)
24  movie_reviews.py:54:0: C0103: Function name "streamAddress" doesn't conform to snake_case naming style (invalid-name)
25  movie_reviews.py:58:0: C0116: Missing function or method docstring (missing-function-docstring)
26
27  -----------------------------------
28  Your code has been rated at 5.47/10
29
30
```

4.issues.md



```
# Static Code Analysis Issues

Line numbers refer to the original `movie_reviews.py` analyzed by the `*_report_before.txt` reports.

## 1. High Severity

### Weak MD5 Hash for Review IDs

- **Tool:** Bandit
- **Code:** B324
- **Lines:** 45–46
- **Problem:** MD5 is a weak hash for security-sensitive use.
- **Fix:** Replace MD5 with SHA-256 if a stronger hash is appropriate. Since this is only a review identifier, document that it is not used for security.

## 2. Medium Severity

### Listening on All Network Interfaces

- **Tool:** Bandit
- **Code:** B104
- **Line:** 4
- **Problem:** `0.0.0.0` allows the service to listen on all network interfaces.
- **Fix:** Use `127.0.0.1` for local-only use, or make the host configurable if external access is required.

## 3. Other Findings

### Hardcoded Moderator Password

- **Tool:** Bandit
- **Code:** B105
- **Line:** 6
- **Problem:** A password is stored directly in source code and may be exposed when the file is shared.
- **Fix:** Remove it if unused, or load the secret from an environment variable or secure configuration.

### Standard Random Generator

- **Tool:** Bandit
- **Code:** B311
- **Line:** 41
- **Problem:** `random.choice` is unsuitable for security-sensitive decisions. In this program it only selects a featured movie, so the selection is not security-sensitive.
- **Fix:** Keep `random` for this non-security use and document the context, or use `secrets.choice` if the selection becomes security-sensitive.

### Missing Module and Function Docstrings

- **Tool:** Pylint
- **Codes:** C0114, C0116
- **Lines:** 1, 11, 16, 26, 34, 39, 45, 48, 54, 58
- **Problem:** The module and functions do not describe their purpose, inputs, or results.
- **Fix:** Add a module docstring and concise docstrings for each function.

### Names Do Not Follow Python Naming Conventions

- **Tool:** Pylint
- **Code:** C0103
- **Lines:** 8, 11, 16, 26, 34, 39, 45, 48, 54; arguments at 16 and 45
- **Problem:** Function and argument names use camelCase rather than snake_case. Pylint also flags `featured` as not following its expected naming style.
- **Fix:** Rename functions and arguments to snake_case, and give the mutable `featured` variable a consistent lower-case name.
```

```
### Unnecessary `else` After `return`

- **Tool:** Pylint
- **Code:** R1705
- **Lines:** 28–30
- **Problem:** The `else` branch is unnecessary because the `if` branch already returns.
- **Fix:** Remove `else` and place the fallback return after the `if` block.

### Global Statement

- **Tool:** Pylint
- **Code:** W0603
- **Line:** 40
- **Problem:** `pickFeatured` changes module-level state with `global`, making the function harder to reason about and test.
- **Fix:** Return the selected title without changing global state, or encapsulate the state in an object if it needs to persist.

### Index Loop Could Use `enumerate`

- **Tool:** Pylint
- **Code:** C0200
- **Lines:** 50–51
- **Problem:** The loop uses `range(len(titles))` to access both an index and a title.
- **Fix:** Iterate with `enumerate(titles, start=1)`.

### Missing Whitespace Around an Operator

- **Tool:** Flake8
- **Code:** E225
- **Line:** 35
- **Problem:** An operator has no surrounding whitespace.
- **Fix:** Add spaces around the operator.

### Missing Blank Line Before a Top-Level Function

- **Tool:** Flake8
- **Code:** E302
- **Line:** 48
- **Problem:** There is not enough blank-line separation before the function definition.
- **Fix:** Add the required blank lines before the function.

### Inline Comment Spacing

- **Tool:** Flake8
- **Code:** E261
- **Line:** 70
- **Problem:** The inline comment does not have enough spaces before it.
- **Fix:** Add at least two spaces before the inline comment.

### Missing Whitespace After a Comma

- **Tool:** Flake8
- **Code:** E231
- **Line:** 72
- **Problem:** A comma is not followed by whitespace.
- **Fix:** Add a space after the comma.
```

Before fixing code

```python
import hashlib
import random

STREAM_HOST = "0.0.0.0"
STREAM_PORT = 8000
MODERATOR_PASSWORD = "reviews-demo"
movies = {}
featured = None


def addMovie(title, genre, year):
    movies[title] = {"genre": genre, "year": year,
                     "ratings": [], "reviews": []}


def rateMovie(title, stars, reviewText=""):
    if stars < 1 or stars > 5:
        print("Rating must be between 1 and 5, got", stars)
        return False
    movies[title]["ratings"].append(stars)
    if reviewText:
        movies[title]["reviews"].append(reviewText)
    return True


def averageRating(title):
    ratings = movies[title]["ratings"]
    if ratings:
        return round(sum(ratings) / len(ratings), 1)
    else:
        return 0.0


def topMovies(count=2):
    ranked=sorted(movies, key=averageRating, reverse=True)
    return ranked[:count]


def pickFeatured():
    global featured
    featured = random.choice(sorted(movies))
    return featured


def reviewId(title, reviewText):
    return hashlib.md5((title + reviewText).encode()).hexdigest()[:10]

def printAllReviews():
    titles = list(movies)
    for i in range(len(titles)):
        print(i + 1, titles[i], averageRating(titles[i]))


def streamAddress():
    return STREAM_HOST + ":" + str(STREAM_PORT)


def main():
    addMovie("The Long Road", "Drama", 2021)
    addMovie("Laugh Track", "Comedy", 2019)
    addMovie("Star Drift", "Sci-Fi", 2023)
    rateMovie("The Long Road", 5, "Beautiful story")
```

```
def printAllReviews():
    titles = list(movies)
    for i in range(len(titles)):
        print(i + 1, titles[i], averageRating(titles[i]))


def streamAddress():
    return STREAM_HOST + ":" + str(STREAM_PORT)


def main():
    addMovie("The Long Road", "Drama", 2021)
    addMovie("Laugh Track", "Comedy", 2019)
    addMovie("Star Drift", "Sci-Fi", 2023)
    rateMovie("The Long Road", 5, "Beautiful story")
    rateMovie("The Long Road", 4, "A bit slow but worth it")
    rateMovie("Laugh Track", 2, "Not very funny")
    rateMovie("Star Drift", 6, "Out of range rating")
    rateMovie("Star Drift", 4)
    printAllReviews()
    print("Top movies:", topMovies())
    print("Featured today:", pickFeatured())
    print("Review id:", reviewId("Laugh Track", "Not very funny")) # demo id
    print("Streaming from", streamAddress())
    print("Genres:",sorted(movie["genre"] for movie in movies.values()))


main()
```


```
def printAllReviews():
    titles = list(movies)
    for i in range(len(titles)):
        print(i + 1, titles[i], averageRating(titles[i]))

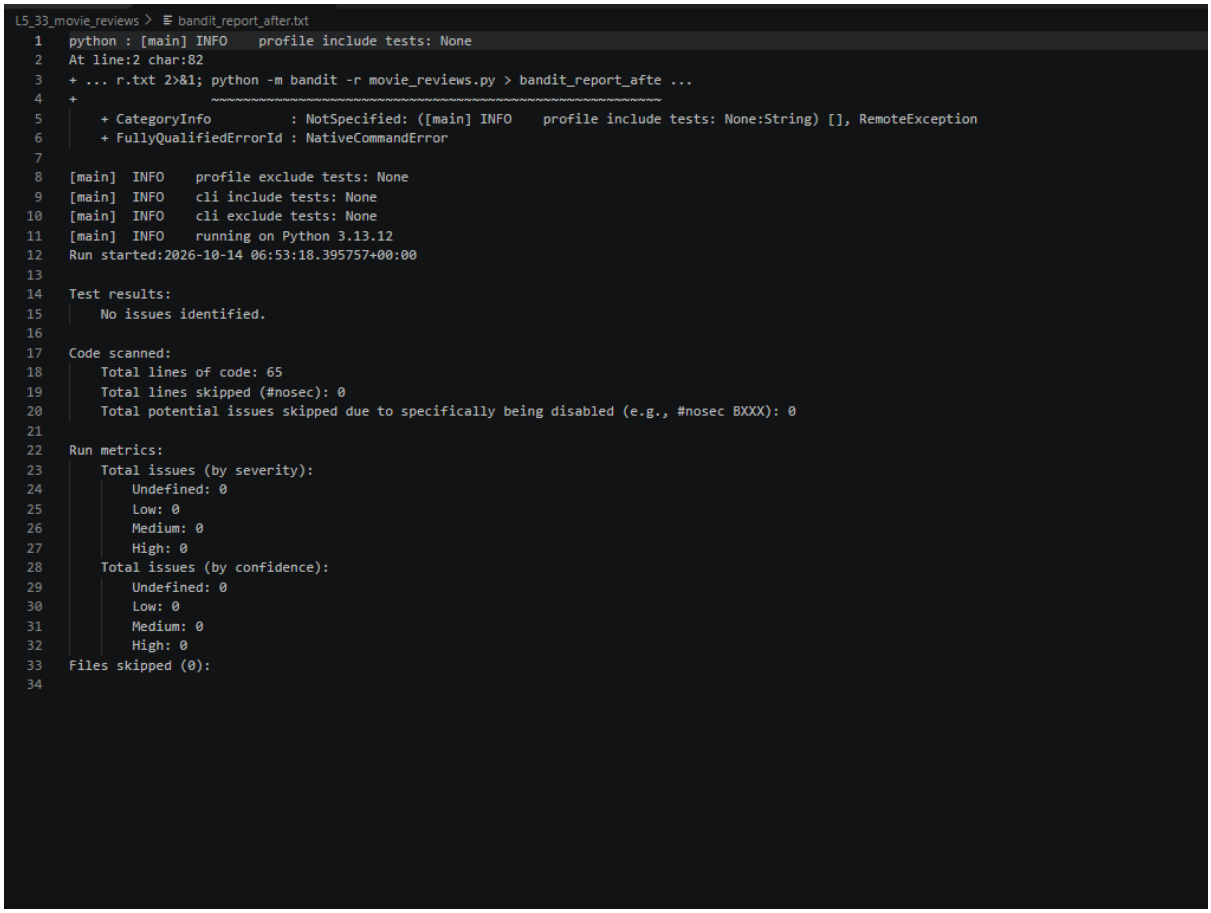

def streamAddress():
    return STREAM_HOST + ":" + str(STREAM_PORT)


def main():
    addMovie("The Long Road", "Drama", 2021)
    addMovie("Laugh Track", "Comedy", 2019)
    addMovie("Star Drift", "Sci-Fi", 2023)
    rateMovie("The Long Road", 5, "Beautiful story")
    rateMovie("The Long Road", 4, "A bit slow but worth it")
    rateMovie("Laugh Track", 2, "Not very funny")
    rateMovie("Star Drift", 6, "Out of range rating")
    rateMovie("Star Drift", 4)
    printAllReviews()
    print("Top movies:", topMovies())
    print("Featured today:", pickFeatured())
    print("Review id:", reviewId("Laugh Track", "Not very funny")) # demo id
    print("Streaming from", streamAddress())
    print("Genres:",sorted(movie["genre"] for movie in movies.values()))

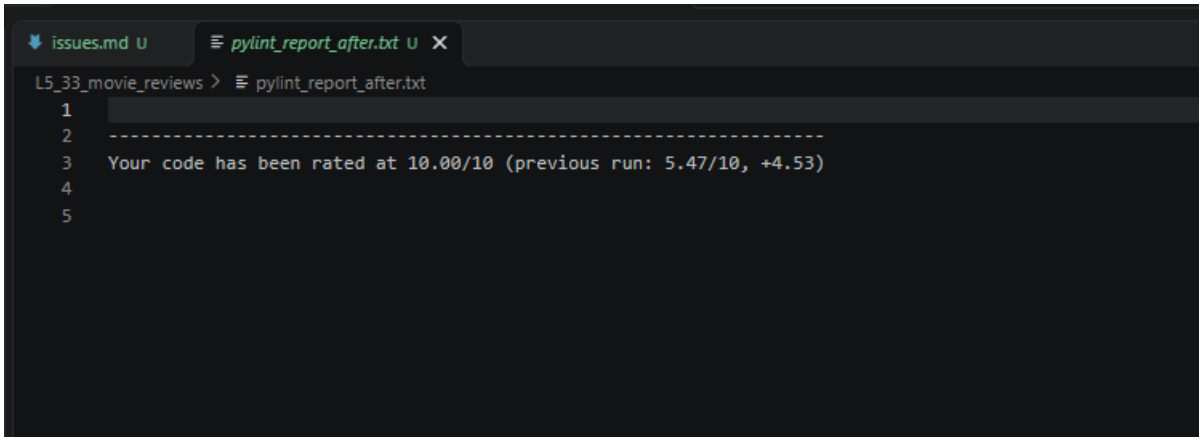

main()
```


AFTER CORRECTION

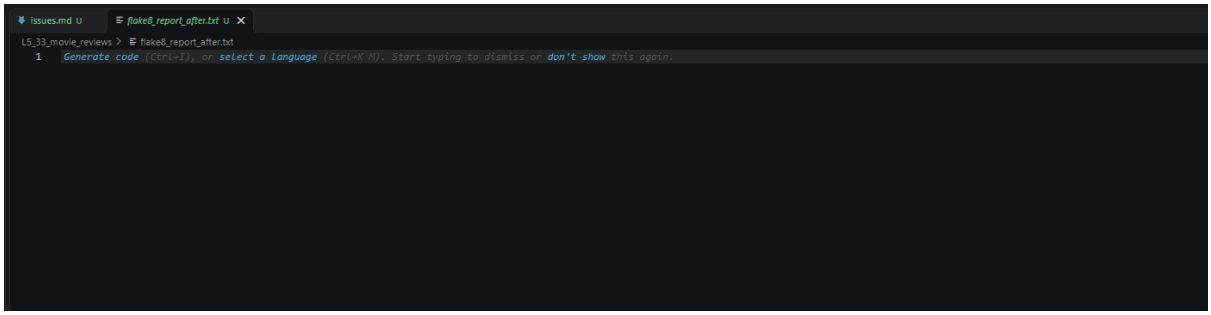
1.Bandit_report



2.pylint



3.flake8



Updated code



L5_33_movie_reviews > movie_reviews.py > ...

```python
def top_movies(count=2):
    """Return up to count movie titles, ranked by average rating."""
    ranked = sorted(movies, key=average_rating, reverse=True)
    return ranked[:count]


def pick_featured():
    """Choose and return a movie title for the featured slot."""
    return secrets.choice(sorted(movies))


def review_id(title, review_text):
    """Return a stable, non-security identifier for a title and review."""
    review_data = (title + review_text).encode()
    return hashlib.sha256(review_data).hexdigest()[:10]


def print_all_reviews():
    """Print each movie title and its average rating."""
    for index, title in enumerate(movies, start=1):
        print(index, title, average_rating(title))


def stream_address():
    """Return the local stream host and port as a string."""
    return f"{STREAM_HOST}:{STREAM_PORT}"
```

```python


def main():
    """Populate sample movies and print the program demonstration."""
    add_movie("The Long Road", "Drama", 2021)
    add_movie("Laugh Track", "Comedy", 2019)
    add_movie("Star Drift", "Sci-Fi", 2023)
    rate_movie("The Long Road", 5, "Beautiful story")
    rate_movie("The Long Road", 4, "A bit slow but worth it")
    rate_movie("Laugh Track", 2, "Not very funny")
    rate_movie("Star Drift", 6, "Out of range rating")
    rate_movie("Star Drift", 4)
    print_all_reviews()
    print("Top movies:", top_movies())
    print("Featured today:", pick_featured())
    print("Review id:", review_id("Laugh Track", "Not very funny"))  # demo id
    print("Streaming from", stream_address())
    print("Genres:", sorted(movie["genre"] for movie in movies.values()))


if __name__ == "__main__":
    main()

```